AULA PARTICULAR · QUARKUS REST

Como resolver o exercício, passo a passo

Este material segue a ordem em que um bom professor ensinaria: primeiro o **porquê**, depois o **como**, depois o **teste**, e só no fim o **gabarito**. Tente cada desafio sozinho antes de olhar a resposta: é assim que se aprende de verdade.

Projeto: quarkus-getting-started · **Guia base:** Guia Quarkus REST (4 exercícios)

---

0 Antes de começar: mapa da aula

PALAVRAS DO PROFESSOR

Os 4 exercícios são uma **escada**: cada degrau usa o anterior. Não pule degraus. Se algo falhar no degrau 3, o erro quase sempre nasceu no degrau 1 ou 2.

#	EXERCÍCIO	O QUE VOCÊ APRENDE	SITUAÇÃO NO SEU PROJETO
1	Product + ProductResource	Criar um endpoint REST com filtro (`@QueryParam`)	**Feito, com `minPrice` também**
2	REST Client (IUserClient, IPostClient)	Consumir uma API externa por interface	**Feito**
3	Configuration (`mp-rest/url`)	Tirar URLs do código e colocá-las em `application.properties`	**URLs feitas · falta o desafio `@ConfigProperty`**
4	JWT + `@RolesAllowed`	Proteger rotas por papel (role)	**NÃO feito: falta tudo (chaves, config, LoginResource, AdminResource)**
+	Extra: log em arquivo	Gravar e ajustar o nível dos logs	**Feito (além do pedido)**

Como as peças se encaixam

```
Navegador/curl ──► ProductResource  (/products)            [Ex.1]
                ├─► UserResource ──► IUserClient ──► jsonplaceholder.typicode.com/users   [Ex.2]
                │                        ▲
                │        application.properties diz a URL base                            [Ex.3]
                └─► LoginResource (gera JWT) ──► AdminResource (@RolesAllowed)            [Ex.4]
```

Ferramentas de que você vai precisar (confira agora)

- ☐ VS Code aberto na pasta `quarkus-getting-started`
- ☐ Java 17+ e Maven (no seu PC: `C:\Users\rafael-ioppi\apache-maven-3.9.9`)
- ☐ Um terminal PowerShell e o programa `curl.exe`
- ☐ OpenSSL (vem junto com o Git for Windows, dentro do **Git Bash**); será usado só no Exercício 4

> **Pegadinha do Windows:** no PowerShell, `curl` é um apelido para `Invoke-WebRequest` e **não aceita** as opções `-X`, `-H`, `-d`. Sempre digite `curl.exe` (com o `.exe`). Todos os comandos deste material já usam assim.

Como subir a aplicação nesta rede (proxy corporativo)

Sua rede exige proxy. O Quarkus não lê o proxy do Windows sozinho, então antes de subir, exporte usuário e senha a partir da variável que o Windows já tem. Faça **em todo terminal novo**:

```
$uri = [uri]$env:HTTPS_PROXY
$env:PROXY_USER = $uri.UserInfo.Split(':')[0]
$env:PROXY_PASSWORD = [uri]::UnescapeDataString($uri.UserInfo.Split(':')[1])
cd "C:\Users\rafael-ioppi\exercicio 1\quarkus-getting-started"
& "C:\Users\rafael-ioppi\apache-maven-3.9.9\bin\mvn.cmd" quarkus:dev
```

Não use `.\mvnw.cmd`: ele tenta baixar o Maven e leva erro 407 no proxy. Quando aparecer `Listening on: http://localhost:8080`, está no ar. **Deixe esse terminal aberto** e use **outro terminal** para os testes com `curl.exe`. O modo dev recarrega o código sozinho ao salvar.

01 Product e ProductResource

Tema: Web Services, expor dados por HTTP em formato JSON.

> **Teoria em 30 segundos.** REST usa os verbos do HTTP (GET, POST, PUT, DELETE) para agir sobre "recursos". No Quarkus, um recurso é uma classe Java com `@Path`. O framework converte HTTP ⇄ Java ⇄ JSON para você.

Passo 1: Modelar os dados com um `record`

Por quê? O produto só carrega dados. Um `record` gera sozinho construtor, getters (`id()`, `name()`...), `equals`, `hashCode` e `toString`, e o Jackson sabe transformá-lo em JSON.

Arquivo: `src/main/java/org/acme/Product.java`

```
package org.acme;

public record Product(Long id, String name, String category, Double price) {
}
```

Passo 2: Criar o recurso REST

Leia cada anotação como uma frase: `@Path("/products")` = "esta classe atende em /products"; `@GET` = "responde a requisições GET"; `@Produces(JSON)` = "devolve JSON"; `@QueryParam("category")` = "pegue o valor que vem depois de `?category=` na URL".

```
@Path("/products")
public class ProductResource {

    private final List<Product> products = List.of(
        new Product(1L, "Product 1", "Category 1", 10.0),
        new Product(2L, "Product 2", "Category 2", 20.0),
        new Product(3L, "Product 3", "Category 3", 300.0));

    @GET
    @Produces(MediaType.APPLICATION_JSON)
    public List<Product> getProducts(@QueryParam("category") String category) {
        if (category == null) {
            return products;
        }
        return products.stream()
            .filter(p -> p.category().equalsIgnoreCase(category))
            .toList();
    }
}
```

Erro clássico: o método precisa ser `public`. O Quarkus ignora em silêncio métodos sem `public` e você recebe 404 sem entender o motivo. Também não confunda `@QueryParam` (lê `?x=`) com `@PathParam` (lê `/{x}`).

Passo 3: Testar

```
curl.exe http://localhost:8080/products
curl.exe "http://localhost:8080/products?category=Category 1"
```

Aspas duplas na URL são obrigatórias quando há `?` ou `&`. Resultado esperado: o primeiro devolve a lista inteira; o segundo, só os produtos da categoria.

Desafio do professor: acrescente um segundo parâmetro, `minPrice`, que devolva só produtos com preço **maior ou igual** ao informado. Teste os dois filtros juntos: `/products?category=Category 3&minPrice=100`.
Dica: cada parâmetro é opcional, então cada `filter` precisa aceitar `null` ("se for nulo, deixa passar").

Meu progresso

- ☐ Sei a diferença entre `@QueryParam` e `@PathParam`
- ☐ Sei por que usar `record`
- ☐ Testei sem filtro e com filtro
- ☐ Resolvi o desafio `minPrice` sozinho

02 REST Client: consumindo uma API externa

Tema: como um serviço conversa com outro pela rede.

> **Ideia central.** Em vez de escrever HTTP "na mão", você **declara uma interface** Java que descreve a API remota. O Quarkus escreve a implementação por você. Você injeta e chama como se fosse um método local. A API usada aqui é a pública `jsonplaceholder.typicode.com`.

Passo 1: Conferir as dependências no `pom.xml`

Devem existir `quarkus-rest-client` e `quarkus-rest-client-jackson`. No seu projeto já estão.

Passo 2: O DTO (o "molde" da resposta)

```
package org.acme;

public record User(Long id, String name, String username, String email) {
}
```

Só declare os campos que quer receber; o JSON traz outros e eles são ignorados.

Passo 3: A interface do cliente

```
@RegisterRestClient
@Path("/users")
public interface IUserClient {

    @GET
    @Produces(MediaType.APPLICATION_JSON)
    List<User> listUsers();
}
```

> **Erro clássico (já ocorreu no seu projeto):** `@RegisterRestClient` vai **somente na interface**, nunca na classe Resource. Note que não há URL aqui: ela virá do Exercício 3.

Passo 4: Injetar e usar

```
@Path("/local-users")
public class UserResource {

    @Inject
    @RestClient
    IUserClient userClient;

    @GET
    @Produces(MediaType.APPLICATION_JSON)
    public List<User> listLocalUsers() {
        return userClient.listUsers();
    }
}
```

`@Inject @RestClient` significa: "Quarkus, me entregue a implementação que você gerou para `IUserClient`".

Passo 5: Testar (só funciona depois do Exercício 3!)

```
curl.exe http://localhost:8080/local-users
```

Esperado: lista de 10 usuários vinda da internet. Se o Exercício 3 ainda não estiver feito, este teste falha: **é normal**.

> **Desafio do professor:** crie `IPostClient` apontando para `/posts`, com `getPost(@PathParam("id") Long id)`, e o endpoint `GET /local-posts/{id}` que devolve o post. Você também precisará do record `Post(Long id, Long userId, String title, String body)`.
> *Teste:* `curl.exe http://localhost:8080/local-posts/1`

Meu progresso

- ☐ Entendi que a interface é só o contrato
- ☐ Sei por que a URL não fica dentro da anotação
- ☐ Testei `/local-users` e recebi a lista
- ☐ Resolvi o desafio `/local-posts/{id}`

03 Configuration: dizendo ao cliente qual URL chamar

Tema: como o Quarkus lê propriedades.

> **Por que fora do código?** Para trocar o endereço entre desenvolvimento, teste e produção sem recompilar. Ordem de prioridade (da maior para a menor): **-D na linha de comando → variável de ambiente → arquivo .env → application.properties**.

Passo 1: Montar a chave

Fórmula do MicroProfile: `{pacote.Interface}/mp-rest/url`. Para `org.acme.IUserClient`:

```
org.acme.IUserClient/mp-rest/url=https://jsonplaceholder.typicode.com
```

Passo 2: Escrever em `src/main/resources/application.properties`

```
org.acme.IUserClient/mp-rest/url=https://jsonplaceholder.typicode.com
org.acme.IPostClient/mp-rest/url=https://jsonplaceholder.typicode.com
```

> **Atenção:** é a URL **base**. O Quarkus soma o `@Path("/users")` da interface. **Nunca** aponte para `localhost:8080`: o cliente chamaria a própria aplicação e falharia.

Passo 3: O proxy da sua rede (específico do seu ambiente)

Como a rede exige proxy, o seu `application.properties` também contém as linhas `quarkus.rest-client."org.acme.IUserClient".proxy-address=...` (e as de `proxy-user` / `proxy-password`, lidas das variáveis `PROXY_USER` / `PROXY_PASSWORD`). A senha **nunca** deve ser escrita no arquivo.

Passo 4: Sobrescrever sem recompilar

```
mvn.cmd quarkus:dev "-Dorg.acme.IUserClient/mp-rest/url=https://outra-api.com/v1"
```

Passo 5: Valores por perfil

O prefixo `%prod.` faz a propriedade valer só em produção (também existem `%dev.` e `%test.`):

```
%prod.org.acme.IUserClient/mp-rest/url=https://api.producao.com/v1
```

Desafio do professor: (a) em `application.properties` crie `app.mensagem-boas-vindas=Ola, aluno!` e uma versão `%prod.app.mensagem-boas-vindas=...`; (b) crie o endpoint `GET /boas-vindas` que devolve esse valor usando `@ConfigProperty`; (c) suba com `-Dapp.mensagem-boas-vindas=Valor da linha de comando` e confirme que o `-D` vence o arquivo.
Import: `org.eclipse.microprofile.config.inject.ConfigProperty`

Meu progresso

- ☐ Sei montar a chave `{pacote.Interface}/mp-rest/url`
- ☐ Sei a ordem de prioridade das fontes de configuração
- ☐ `/local-users` e `/local-posts/1` funcionam de verdade
- ☐ Resolvi o desafio `/boas-vindas`

04 Segurança com JWT e @RolesAllowed

Tema: autenticação e autorização por papéis. ***Este é o exercício que ainda falta no seu projeto.***

> **Analogia: a pulseira do festival.** Você mostra o documento uma vez na entrada (**login**) e recebe uma pulseira (**token JWT**). Em cada área (**endpoint**), o segurança só olha a pulseira: ela é verificável e diz quais áreas você pode entrar (**roles**).
> Um JWT tem 3 partes separadas por ponto: **header** (algoritmo) · **payload** (usuário e roles, chamadas de `groups`) · **signature** (garante que ninguém alterou o conteúdo).

Passo 1: Dependências

Já estão no seu pom.xml (`quarkus-smallrye-jwt` e `quarkus-smallrye-jwt-build`). Nada a fazer.

Passo 2: Gerar o par de chaves RSA

Por quê? A chave **privada assina** o token (fica só com o servidor). A chave **pública valida** (pode ser distribuída). Abra o **Git Bash** na pasta do projeto:

```
cd "/c/Users/rafael-ioppi/exercicio 1/quarkus-getting-started/src/main/resources"
openssl genrsa -out rsaPrivateKey.pem 2048
openssl pkcs8 -topk8 -nocrypt -inform pem -in rsaPrivateKey.pem -outform pem -out privateKey.pem
openssl rsa -in rsaPrivateKey.pem -pubout -out publicKey.pem
rm rsaPrivateKey.pem
```

Resultado: os arquivos `privateKey.pem` e `publicKey.pem` dentro de `src/main/resources`. (A conversão pkcs8 evita um erro de formato de chave em versões novas do OpenSSL.)

> **Segurança:** a chave privada **nunca** vai para um repositório público. Este é um exercício; em produção ela ficaria fora do projeto.

Passo 3: Configurar assinatura e validação

Acrescente ao final do `application.properties`:

```
# Assinatura (quem GERA o token: LoginResource)
smallrye.jwt.sign.key.location=privateKey.pem

# Validacao (quem RECEBE o token: @RolesAllowed)
mp.jwt.verify.issuer=https://acme.org/issuer
mp.jwt.verify.publickey.location=publicKey.pem
```

O `issuer` aqui e o usado no código do Passo 4 precisam ser **exatamente a mesma string**.

Passo 4: Endpoint que gera o token (login)

Arquivo: `src/main/java/org/acme/LoginResource.java`. `@PermitAll` libera o endpoint sem token (afinal, é aqui que você ainda não o tem).

```
package org.acme;

import java.util.HashSet;
import java.util.Set;

import io.smallrye.jwt.build.Jwt;
import jakarta.annotation.security.PermitAll;
import jakarta.ws.rs.POST;
import jakarta.ws.rs.Path;
import jakarta.ws.rs.Produces;
import jakarta.ws.rs.core.MediaType;

@Path("/login")
public class LoginResource {

    private static final String ISSUER = "https://acme.org/issuer";

    @POST
    @PermitAll
    @Produces(MediaType.TEXT_PLAIN)
    public String generate(final String username) {
        Set<String> roles = new HashSet<>();
        roles.add("admin");
        return Jwt.issuer(ISSUER)
            .upn(username)
            .groups(roles)
            .expiresIn(3600)
            .sign();
    }
}
```

Passo 5: Endpoint protegido

Arquivo: `AdminResource.java`. Com `@RolesAllowed("admin")`, o Quarkus intercepta a chamada **antes** do seu código: lê o header `Authorization: Bearer <token>`, valida a assinatura com a chave pública e confere se `"admin"` está em `groups`. Você não escreve nenhum `if`.

```
package org.acme;

import jakarta.annotation.security.RolesAllowed;
import jakarta.ws.rs.GET;
import jakarta.ws.rs.Path;
import jakarta.ws.rs.Produces;
import jakarta.ws.rs.core.MediaType;

@Path("/admin")
public class AdminResource {

    @GET
    @RolesAllowed("admin")
    @Produces(MediaType.TEXT_PLAIN)
    public String adminOnly() {
        return "Acesso liberado: voce tem a role admin";
    }
}
```

Passo 6: Testar os três cenários (PowerShell)

CENÁRIO	COMANDO	ESPERADO
1. Gerar token	`$TOKEN = curl.exe -s -X POST -d "rafael" http://localhost:8080/login`	200 + texto longo (o JWT)
2. Sem token	`curl.exe -i http://localhost:8080/admin`	**401** Unauthorized
3. Com token	`curl.exe -H "Authorization: Bearer $TOKEN" http://localhost:8080/admin`	**200** + mensagem

Dica: cole o token em **jwt.io** para ver o payload. Você verá `upn: rafael` e `groups: ["admin"]`.

Quando der errado (tabela de diagnóstico)

SINTOMA	CAUSA PROVÁVEL	SOLUÇÃO
401 mesmo com token	Faltou `Bearer ` (com espaço) no header	Enviar exatamente `Bearer <token>`
401 logo após gerar	`mp.jwt.verify.issuer` diferente do `ISSUER` do código	Deixar as duas strings idênticas
403 Forbidden	Token válido, mas sem a role exigida	Conferir `.groups(roles)`
Erro ao iniciar citando "key location"	.pem fora de `src/main/resources` ou mal gerado	Refazer o Passo 2
`curl` reclama de parâmetro	Usou `curl` em vez de `curl.exe` no PowerShell	Usar `curl.exe`

Desafio do professor: crie a role `gerente` e o endpoint `GET /relatorios` protegido com `@RolesAllowed({"admin","gerente"})`. Gere um token **só com** `gerente` e confirme: acessa `/relatorios` (200) mas leva **403** em `/admin`.
Dica: hoje o `LoginResource` sempre põe `admin`. Faça o papel depender de um dado recebido (por exemplo, criar um segundo endpoint `/login-gerente`, ou ler o nome do usuário).

Meu progresso

- ☐ Sei as 3 partes de um JWT
- ☐ Sei a diferença entre chave privada (assina) e pública (valida)
- ☐ Testei: login → token → acesso 200
- ☐ Testei a falha: sem token → 401
- ☐ Resolvi o desafio `gerente` (200 em /relatorios, 403 em /admin)

+ Extra: log em arquivo (além do exercício)

*Tema: acompanhar o que a aplicação está fazendo. Isto **não** era pedido no guia; foi um acréscimo feito no seu projeto.*

> **Para que serve?** Por padrão o Quarkus só escreve o log no terminal, que se perde ao fechá-lo. Com estas propriedades, tudo também é gravado em um arquivo, e as mensagens do seu próprio pacote (`org.acme`) aparecem com mais detalhe (nível `DEBUG`).

Passo 1: Acrescentar ao `application.properties`

```
quarkus.log.file.enable=true
quarkus.log.file.path=target/quarkus.log
quarkus.log.level=INFO
quarkus.log.category."org.acme".level=DEBUG
```

PROPRIEDADE	O QUE FAZ
`quarkus.log.file.enable`	Liga a gravação do log em arquivo
`quarkus.log.file.path`	Onde o arquivo fica (aqui, `target/quarkus.log`)
`quarkus.log.level`	Nível geral: `INFO` mostra o essencial de todas as bibliotecas
`quarkus.log.category."org.acme".level`	Nível só para o seu código: `DEBUG` mostra mais detalhes

Passo 2: Testar

Suba a aplicação, faça algumas chamadas (por exemplo `curl.exe http://localhost:8080/products`) e abra o arquivo:

```
Get-Content "C:\Users\rafael-ioppi\exercicio 1\quarkus-getting-started\target\quarkus.log" -Tail 20
```

> **Atenção:** o arquivo fica dentro de `target/`, pasta apagada por `mvn clean`. É normal: o log é recriado na próxima execução.

Meu progresso

- ☐ Sei a diferença entre o nível geral e o nível por pacote
- ☐ Encontrei e li o arquivo `target/quarkus.log`

★ Gabarito dos desafios (só depois de tentar!)

PALAVRAS DO PROFESSOR

Errar e depurar faz parte do aprendizado. Compare a sua solução com esta só depois de ter tentado por, no mínimo, 15 minutos.

Ex. 1: filtro por `minPrice`

```
@GET
@Produces(MediaType.APPLICATION_JSON)
public List<Product> getProducts(@QueryParam("category") String category,
                                 @QueryParam("minPrice") Double minPrice) {
    return products.stream()
        .filter(p -> category == null || p.category().equalsIgnoreCase(category))
        .filter(p -> minPrice == null || p.price() >= minPrice)
        .toList();
}
```

Ex. 2: IPostClient e /local-posts/{id}

```
@RegisterRestClient
@Path("/posts")
public interface IPostClient {
    @GET
    @Path("/{id}")
    @Produces(MediaType.APPLICATION_JSON)
    Post getPost(@PathParam("id") Long id);
}

@Path("/local-posts")
public class PostResource {
    @Inject @RestClient IPostClient postClient;

    @GET
    @Path("/{id}")
    @Produces(MediaType.APPLICATION_JSON)
    public Post getLocalPost(@PathParam("id") Long id) {
        return postClient.getPost(id);
    }
}
```

Ex. 3: /boas-vindas com @ConfigProperty

```
# application.properties
app.mensagem-boas-vindas=Ola, aluno!
%prod.app.mensagem-boas-vindas=Bem-vindo a producao!

// BoasVindasResource.java
@Path("/boas-vindas")
public class BoasVindasResource {

    @ConfigProperty(name = "app.mensagem-boas-vindas")
    String mensagem;

    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String get() {
        return mensagem;
    }
}
```

Teste com `-Dapp.mensagem-boas-vindas="Valor da linha de comando"`: o `-D` vence o arquivo, confirmando a ordem de prioridade.

Ex. 4: role gerente

```
// LoginGerenteResource.java (classe nova, nao mexe no LoginResource)
@Path("/login-gerente")
public class LoginGerenteResource {
    @POST
    @PermitAll
    @Produces(MediaType.TEXT_PLAIN)
    public String generate(final String username) {
        return Jwt.issuer("https://acme.org/issuer").upn(username)
            .groups(Set.of("gerente")).expiresIn(3600).sign();
    }
}

// RelatorioResource.java
@Path("/relatorios")
public class RelatorioResource {
    @GET
    @RolesAllowed({"admin", "gerente"})
    @Produces(MediaType.TEXT_PLAIN)
    public String relatorio() { return "Relatorio confidencial"; }
}
```

TESTE	RESULTADO ESPERADO
Token gerente → `/relatorios`	**200**
Token gerente → `/admin`	**403**
Token admin → `/relatorios` e `/admin`	**200 nos dois**
Sem token → qualquer um dos dois	**401**

✓ Revisão final e anotações

Cinco perguntas de prova (responda sem consultar)

1) Qual a diferença entre `@QueryParam` e `@PathParam`?

---

2) Por que `@RegisterRestClient` vai na interface e a URL vai no `application.properties`?

---

3) Em que ordem o Quarkus procura uma propriedade (-D, variável de ambiente, .env, application.properties)?

---

4) Qual a diferença entre erro 401 e 403?

---

5) Por que a chave privada assina e a pública valida, e não o contrário?

▼ Respostas curtas

1) `@QueryParam` lê `?x=1` na URL; `@PathParam` lê o pedaço do caminho `/{id}`. 2) A interface é o contrato e a URL muda por ambiente, então fica fora do código. 3) -D → variável de ambiente → .env → application.properties. 4) 401 = não autenticado (sem token válido); 403 = autenticado, mas sem permissão (role errada). 5) Só o servidor pode criar tokens (privada, secreta), mas qualquer serviço deve poder conferi-los (pública, distribuível).